

COMP 364/464

Project 3: HPC Project

Term: Fall 2026

Instructor: Melanie Cornelius

Assigned: 12 October 2026

Check-in: 28 October 2026, 11:59 PM AoE

Due: Final exam date (TBA), 11:59 PM AoE

Description

This is the capstone project for the semester. You'll pick something HPC-related, make the case that it's HPC, build or run it, monitor it, and write it up. The project is open-ended; you are encouraged to modify work you're currently doing or try things that end up not working! If something catastrophically fails, simply explain what happened and what you learned from it.

You are welcome to work individually or in a pair. Pairwise teams will be expected to produce more results compared to individual projects.

You should plan on spending 1-3 hours a week on this for the rest of the semester. You will have at least one week of course time to work during the typically scheduled lecture.

Before You Start

Linux is required. You'll need `/proc` and `perf` for the monitoring part of this project, so if you're on macOS or Windows, use the Linux setup from Project 2 Part B (Multipass or WSL2).

Containers (Docker, Podman, etc.) are optional and not encouraged unless your project is specifically concerning virtualization. They're uncommon on large HPC systems, and I'd rather you build and run your software natively. If you have a good reason to use them, though, explain it in your plan.

Choosing a Project

Your project needs to meet at least two of the following (464: you should meet all three or have a strong alignment with two):

- **Measured parallelism:** your work runs across cores, processes, nodes, or GPUs, and you measure how and why it scales.
- **Performance against a reference:** you compare your measurements to a model or a known HPC benchmark, such as memory bandwidth from STREAM, a roofline model, theoretical peak, or other published benchmark results.
- **A real HPC system component:** you install, configure, simulate, or study something HPC centers actually run, such as a job scheduler simulator, a parallel file system, an interconnect, a monitoring stack, or a checkpointing tool.

Every project, regardless of topic, must also include:

- **Monitoring:** watch your work while it runs and use what you collect to explain your results. Wall time alone isn't enough! You can use the `/proc` and `perf` tools from class, GPU tools from your vendor, I/O or network monitoring, or a profiling tool we haven't covered. You should be able to point at your monitoring data and say "this is why the result looks like this."
- **The scale:** explain how your project differs from what you might see at (or what is published on) a large HPC center. What would change at 1,000 or 10,000 nodes? What couldn't you test on your own hardware?

Some ideas to get you started (this is not a complete list):

- **Scale a computation:** parallelize a problem with MPI, OpenMP, CUDA (or another GPU programming model), or a combination, and run a scaling study.
- **Benchmark a machine:** run several (micro)benchmarks like STREAM, HPL, HPCG, OSU Micro-Benchmarks, or IOR, explain what each one measures, and compare your local results to published numbers.
- **Go deeper on monitoring:** use a performance tool beyond what we did in class (for example LIKWID, HPCToolkit, TAU, or Score-P) to find and explain a performance problem in a program. *This is a great option if you're doing research in another area!*
- **Simulate a scheduler:** use a scheduler simulator or a small local scheduler install, and compare scheduling policies. This one is complex, but many simulators and sample datasets exist (CQSim, PBS, Slurm).
- **Simulate a network:** use a network simulator to compare topologies we covered in class (torus vs. dragonfly, for example) under different traffic patterns. This is a fun one if you like networks!
- **Set up a shared file system:** install a shared or parallel file system across VMs or machines and benchmark it. This is probably the easiest option, though setting up multiple VMs will require your local hardware to be able to handle it.

If you're stuck choosing a topic, or you aren't sure your topic works, reach out on Piazza or come to office hours. Please don't wait until the check-in to find out your idea needs changes!

Individual or Teams

You may work alone or with one partner. If you work in a pair, I'll expect a bit more: a larger scaling study, an additional HPC criterion, or a broader scope. Describe what the extra scope is in your plan. Pairs must include a short statement of who did what.

Check-in (28 October)

Submit:

- **A project plan (1 to 2 pages):** what you'll build or run, which HPC criteria it meets and why, what you'll monitor and how, what hardware you're using, and what you'll do if an install or tool doesn't work
- **Drafts:** whatever you have so far, such as early code, a first run, a first measurement, or a report outline
- I'll be looking for you to have just gotten started, don't worry if you don't have much to show yet!

Final Report

Your report should include:

- Why it's HPC: make the case for each criterion you claim
- What you did and why
- Your setup: hardware, software and versions, and how you ran things
- A description of your files, scripts, datasets, and any other component I'd need to replicate your results
- Your results, with your monitoring data used to explain them
- The scale difference from your local machine to HPC
- What went wrong or surprised you
- References (make sure they resolve!)

464 Only - Real-World Comparison: In addition to meeting three criteria, compare your project to how a real HPC center does the same thing using documentation, papers, or user guides. Reach out if you want a hand with this!

Deliverables:

- Your report, including the plan from the check-in (updated if your plan changed)
- Source code and a Makefile or build script
- All scripts you used to collect data, with a comment at the top saying what each one collects
- Raw data, plots, and any monitoring output
- A README with exact steps to build and run your project on a clean Linux install, including all dependencies and versions
- A single script or Makefile target that runs your project. If your code takes a long time to run (>2 hours), also provide an option to run a subset so I can tell if it worked
- Documentation of the full-size runs used in your report

Grading:

- Check-in (plan and drafts): 10%
- Report: (65% total)
 - HPC justification: 10%
 - Experiments and results: 25%
 - Monitoring and explanations: 20%
 - Scale to HPC (and real-world comparison for 464): 10%
- Code, scripts, README, and run package: 25%

The same academic honesty and AI policies apply. If you use AI tools, document your process and model usage. You should be able to explain every number in your report and every line of every script.

Turn in the check-in and the final project as a .zip or .tar on Sakai.